

Real-time Airfare Price Index for India — Project Flow & Presentation Prompt

Problem Statement 26056 | MoSPI | Smart Automation

A. Simple System Flow

1. Data Sources	Airline portals + OTAs → scheduled collection of route, date, carrier, fare class, base fare, taxes and total fare.
2. Ethical Scraping	Python Playwright/Scrapy/Selenium → respect robots.txt/ToS, rate limits, session controls and permitted access. Do not bypass CAPTCHAs or access controls.
3. Data Pipeline	Raw quotes → validation → deduplication → missing-value handling → outlier treatment → sold-out/cancellation flags → fare-component separation.
4. Database	Firebase/Firestore for prototype metadata and dashboard data; store timestamp, route, advance window, carrier, fare class and price.
5. Index Engine	Representative city-pairs + DGCA/approved weights → route-level price relatives → weighted aggregate Airfare Price Index (daily/weekly/monthly).
6. Backend/API	Python service exposes cleaned observations, index values and trend endpoints. Keep scraping/processing separate from React UI.
7. React Dashboard	React + TypeScript + Tailwind → KPI cards, India/route heatmap, sector trends, lead-time elasticity, carrier comparison and data-quality status.
8. Validation	Back-test at least 30 days against publicly available DGCA monthly average-fare data; report MAE/MAPE/correlation and limitations.

Architecture: Sources → Scraping Scheduler → Raw Data → Cleaning/Normalisation → Firestore/Database → Index Calculation → API → React/Tailwind Dashboard → Reports/API consumers

Recommended prototype stack: React + TypeScript + Tailwind CSS + Firebase/Firestore + Python FastAPI/Flask + Playwright/Scrapy + scheduled jobs. Firebase is excellent for the prototype dashboard, but heavy scraping and index computation should run in a backend/worker, not in React.

Important compliance point: design the collector to use permitted/publicly accessible data and published APIs/feeds where available. Do not implement CAPTCHA bypass, credential abuse, or evasion of access controls.

B. One-Page Presentation / Slide Prompt

01 — Problem & Why Now — India's airfare is dynamic; manual CPI collection misses online, route-specific, time-sensitive prices. State the 200–400% intraday variation as a problem-statement claim to be validated, not as an assumed result.

02 — Proposed Solution — Real-time Airfare Price Index (APIx): automated collection → cleaning → normalisation → weighted index → dashboard/API.

03 — Data Sources & Basket — Airlines + OTAs; representative routes such as DEL-BOM, DEL-BLR, BOM-BLR, DEL-CCU, BLR-HYD, MAA-DEL; T+1/T+7/T+15/T+30/T+45.

04 — System Architecture — Show the 8-step flow from source collection to React dashboard and API.

05 — Data Cleaning & Normalisation — Show raw quote → validation → outlier/missing handling → base fare + taxes + charges → canonical quote.

06 — Index Methodology — Explain route weights, price relatives, aggregation frequency, and how the index is rebased. Include a small worked example.

07 — Dashboard UI — React/Tailwind screens: CPI/APIx KPI, daily trend, route heatmap, lead-time curve, carrier comparison, filters and data freshness.

08 — Validation & Back-test — 30-day comparison with DGCA data; show MAE/MAPE/correlation, route coverage and limitations.

09 — Impact & API — How NSO/RBI can consume the index; reproducibility, audit trail, scheduled updates and export/API access.

10 — Demo & Future Scope — Live route search/index update, alerts, broader city basket, improved weighting, additional permitted data sources.

Design prompt for React/Tailwind: “Build a professional government-data analytics dashboard for India's Real-time Airfare Price Index. Use a clean white/navy/blue data-visualisation style, responsive cards, accessible typography, route filters, KPI cards, line

charts, heatmap, lead-time elasticity chart, carrier comparison, data freshness badge, methodology drawer and downloadable report/API sections. Keep the UI evidence-driven and avoid decorative elements that hide data."